TODO: Thesis Title

by

Andrew M. Spears

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

May 2026

Authored by: Andrew M. Spears
Department of Electrical Engineering and Computer Science
TODO: submission date

Certified by: Adam Chlipala
Professor of Computer Science, Thesis Supervisor

Accepted by: Leslie A. Kolodziejski
Professor of Electrical Engineering and Computer Science
Chair, Department Committee on Graduate Students

TODO: Thesis Title

by

Andrew M. Spears

Submitted to the Department of Electrical Engineering and Computer Science
on TODO: submission date in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

ABSTRACT

TODO: abstract

Thesis supervisor: Adam Chlipala
Title: Professor of Computer Science

Contents

List of Figures

List of Tables

Chapter 3

Register Refactoring

3.1 Register Types

Fiat Cryptography was originally designed around scalar x86 registers, and therefore had no notion of registers larger than 64 bits. AVX includes 128-bit xmm and 256-bit ymm registers, so we needed to refactor the entire register system to accomodate these. The simplest change began with the register type. Scalar and vector registers are fundamentally different in terms of how they interact with the rest of the codebase, in a few important ways:

- Only scalar registers can form memory-address operands
- Only scalar registers can participate in flag-setting arithmetic and carry chains
- Only scalar registers can appear in calling conventions
- Scalar registers' alias hierarchy is four levels deep (al/ax/eax/rax)
- Vector registers carry opaque packed-lane data
- Vector registers alias only two levels deep (xmm/ymm)

we made the two register kinds distinguishable by type, with a unifying type REG = SReg SREG | VReg VREG.

```
1  (* Unified register type: scalar or vector *)
2  Inductive REG :=
3  | SReg (r : SREG)
4  | VReg (v : VREG)
5  .
```

Downstream functions which need to distinguish between the two can now do so directly by type. For example, a memory location needs to accept only scalar registers for the base address:

```
Record MEM := {
  mem_bits_access_size : option AccessSize ;
  mem_base_reg : option SREG ;
  mem_scale_reg : option (Z * SREG) ;
  mem_base_label : option string ;
  mem_offset : option Z ;
  rip_relative : rip_relative_kind
}.
```

3.2 Register get/set

Many existing functions for interacting with register values in Fiat-Crypto were specialized to handling 64-bit values. For example, the SetReg function uses two branches for 64-bit writes and smaller register writes because an 8 or 32-bit write neesd to combine the new value iwth the existing value, whereas a 64-bit write can overwrite hte old value, meaning the old value can be completely forgotten (or be unspecified to begin with), simplifying the computation graph in many cases.

```
Definition SetReg {opts : symbolic_options_computed_opt} {descr:description}
  r (v : idx) : M unit :=
  let '(rn, lo, sz) := index_and_shift_and_bitcount_of_reg r in
  if N.eqb sz 64
  then v <- App (slice 0 64, [v]);
      SetReg64 rn v (* works even if old value is unspecified *)
  else old <- GetReg64 rn;
      v <- App ((set_slice lo sz), [old; v]);
      SetReg64 rn v.
```

However, when writing to a 128-bit wide xmm register, even a 64-bit write needs to set a slice of the old value, so this function should check more generally if the write matches the largest size of this register's aliasing hierarchy:

```
Definition SetReg {opts : symbolic_options_computed_opt} {descr:description}
  r (v : idx) : M unit :=
  let '(rn, lo, sz) := index_and_shift_and_bitcount_of_reg r in
  (* check size against widest register in heirarchy *)
  if (N.eqb lo 0) && (N.eqb sz (widest_reg_size_of r))
  then v <- App (slice 0 sz, [v]);
       SetRegFull rn v (* works if old value is unspecified *)
  else old <- GetRegFull rn;
       v <- App ((set_slice lo sz), [old; v]);
       SetRegFull rn v.
```

This kind of refactor requires new lemma structure as well. In this case, we introduced two new lemmas:

```
Lemma R_SetReg_full {opts : symbolic_options_computed_opt} {descr:description}
  s m (HR : R s m) r i _tt s'
  (Hoffset : reg_offset r = 0%N)
  (Hwidest : reg_size r = widest_reg_size_of r)
  (H : Symbolic.SetReg r i s = Success (_tt, s'))
  v (Hv : eval s i v)
  : exists m', Some (update_reg_with m (fun rs => set_reg rs r v)) = Some m'
    /\ R s' m' /\ s :< s'.
```

```
Lemma R_SetReg_partial {opts : symbolic_options_computed_opt} {descr:description}
  s m (HR : R s m) r i _tt s'
  (Hpartial : ((reg_offset r =? 0)%N && (reg_size r =? widest_reg_size_of r)%N)%bool = false)
  (H : Symbolic.SetReg r i s = Success (_tt, s'))
  v (Hv : eval s i v)
  : exists m', Some (update_reg_with m (fun rs => set_reg rs r v)) = Some m'
  /\ R s' m' /\ s :< s'.
```

Chapter 4

Memory Refactoring

Like registers, a lot of memory operations were originally built around 64-bit values. Memory in Fiat-Crypto is represented in two ways. In semantics, memory state is a mapping from 64-bit addresses to bytes:

```
1  Definition mem_state := (SortedListWord.map (Naive.word 64) Byte.byte).
```

This represents what is actually stored in the machine. On the other hand, we have a representation which is referenced when memory operations are added to the symbolic execution graph. This representation only needs to store values in abstract to keep track of how each store operation influences later load operations, so it uses a mapping between symbolic execution nodes:

```
1  Definition mem_state := list (idx * idx).
```

where both keys and values are implicitly both 64-bits. This is important because there is no overflow from one memory write to the next address - writing to address a cannot influence address a+8 in this model, so the values need to be a consistent size. One could have used bytes like the semantic memory model, but symbolic execution is far simpler when eveyrthing uses 64 bits, the most common size. Thus, two operations exist for this purpose:

```
1  Definition Load64 (a : idx) : M idx :=
2    some_or (load a) (error.load a).
```

```
Definition Store64 (a v : idx) : M unit :=
  ms <- some_or (store a v) (error.store a v);
```

Then for loading a single Load function handles the sizes of values involved:

```
(* Old *)
Definition Load {opts : symbolic_options_computed_opt} {descr:description}
  {s : OperationSize} {sa : AddressSize} (a : MEM) : M idx :=
  (* Error if unsupported memory access size *)
  if negb (orb (Syntax.operand_size a s =? 8 )( Syntax.operand_size a s =? 64))%N
  then err (error.unsupported_memory_access_size (Syntax.operand_size a s))
  else
  addr <- Address a;
  v <- Load64 addr;
  App ((slice 0 (Syntax.operand_size a s)), [v]).
```

Storing is handled similarly.

Some aspects of this model should stay fixed. Memory addresses should always be 64-bits, and it makes sense to keep these data structures. But we often want to store or load chunks of more than 64 bits when working with AVX registers. For example, `vmovdqu ymm0, [rbx]` loads 32 bytes of memory into the ymm0 register.

To address this, we need to change the existing Load and Store functions to allow larger memory reads without altering the 64-bit value memory model. The simplest approach is to break larger operations into 64-bit chunks:

```
Fixpoint Load_of_idx {opts : symbolic_options_computed_opt} {descr:description}
  {sa : AddressSize} (n : nat) (addr : idx) : M idx :=
  match n with
  | O      => App (const 0, nil)
  | S n'   => prev   <- Load_of_idx n' addr;
              offset <- App (const (8 * Z.of_nat n'), nil);
              addr_k <- App (add sa, [addr; offset]);
              chunk  <- Load64 addr_k;
              App (set_slice (64 * N.of_nat n') 64, [prev; chunk])
  end.
```

```
Definition Load {opts : symbolic_options_computed_opt} {descr:description}
  {s : OperationSize} {sa : AddressSize} (a : MEM) : M idx :=
  let sz := Syntax.operand_size a s in
  addr <- Address a;
  if ((sz =? 8) || (sz =? 64))%N%bool then
    v <- Load64 addr;
    App ((slice 0 sz), [v])
  else if (sz =? 128)%N then
    Load128_of_idx addr
  else if (sz =? 256)%N then
    Load256_of_idx addr
  else err (error.unsupported_memory_access_size sz).
```

This approach retains the simplicity of 64-bit loads (notice the first branch calls Load64 directly). We keep the memory model unchanged, but larger memory operations pay the price of lots of extra operations to compute offset, slicing, and loading individual chunks.

To prove correctness of this new structure, we need to prove that the `Load_of_idx` function (affects symbolic state) is equivalent to loading the appropriate number of 8-byte chunks with `get_mem` (affects machine state). We can do this with the following lemma:

```
(* General (n*64)-bit load. Load128_R / Load256_R are corollaries below. *)
Lemma LoadN_R {opts : symbolic_options_computed_opt} {descr:description}
  {sa : AddressSize}
  (Hsa : sa = 64%N)
  n s m (HR : R s m) (addr : idx)
  va (Ha : eval s addr va)
  i s' (H : Load_of_idx n addr s = Success (i, s'))
  : R s' m /\ s :< s' /\
    exists v, eval s' i v /\
              get_mem m va (8 * n) = Some v /\
              v = Z.land v (Z.ones (64 * Z.of_nat n)).
```

which admits a proof by induction on n.

```
1  Lemma get_mem_app_split m va (chunks : nat) v1 v2 :
2    get_mem m va (8*chunks) = Some v1 ->
3    get_mem m (Z.land (va + Z.of_nat (8*chunks)) (Z.ones 64)) 8 = Some v2 ->
4    get_mem m va (8*chunks + 8) = Some (Z.lor v1 (Z.shiftl v2 (64 * Z.of_nat chunks))).
```

include this?

Chapter 5

Symbolic Execution and Rewriting for Vector Instructions

When an instruction is symbolically executed, we need a way of interpreting what computation is actually being done. There are 2 principled ways to do this for any (nontrivial) instruction, each of which abstracts details to one place or another. The first way is to break the instruction down into a composition of simpler operations as soon as it is symbolically executed, and the second is to include a single atomic operation in the execution graph.

The first approach breaks vector instructions into their component operations in symbolic execution. Most vector instructions follow the same pattern: slice lane from both source registers, perform a binary operation, write to a lane of the destination register. For example, the `vpaddq` (vector packed addition) instruction performs a 64-bit addition on each lane of two 256-bit YMM registers and writes the results to the corresponding lanes of the destination register. This entire instruction can be expressed as a composition of simpler operations: slicing, scalar addition, and `set_slice` to combine results back into a vector.

This pattern can be captured by a few helper functions. The function doing the heavy-lifting takes a scalar binary operation and applies it to each lane of the input vectors, building up the result with `set_slice` operations:

```
Fixpoint vector_binop_aux {opts : symbolic_options_computed_opt} {descr : description}
    (v1 v2 : idx) (lane_op : op) (lane_idx : nat) (num_remaining : nat)
    (lane_width : Z) (acc : idx) : M idx :=
    match num_remaining with
    | O => ret acc
    | S n =>
        lane_val <- make_lane v1 v2 lane_op lane_idx lane_width;
        new_acc <- App (
            set_slice (N.of_nat lane_idx * Z.to_N lane_width) (Z.to_N lane_width),
            [acc; lane_val]);
        vector_binop_aux v1 v2 lane_op (S lane_idx) n lane_width new_acc
    end.
```

This makes the symbolic execution of most instructions simple to implement. For example, `vpaddq` can be implemented as follows:

```
Definition SymexNormalInstruction {opts : symbolic_options_computed_opt} {descr:description}
    (instr : NormalInstruction) : M unit :=
    match instr.(Syntax.op), instr.(args) with
    ...
    | vpaddq, [dst; src1; src2] => (* packed add of quadwords - no flags affected *)
        SymbolicVector.SymexVectorBinOp dst src1 src2 (add 64) 4 64
    ...
```

But this approach makes the symbolic execution graph much more complex, because a 4-lane vector instruction uses roughly 12 nodes to slice inputs, perform lane ops, and slice the output together. This has two downsides: First, canonicalizing the execution graph becomes more complex. We can easily end up with towers of nested slices and `set_slice` deep in the graph, and we need powerful rewrite rules to recognize patterns and read many layers into existing nodes to find them. Second, the execution graph has lost information that might be useful for other reasons.

expand

The second approach keeps symbolic execution simple, emitting a single atomic `op` for each vector instruction and abstracting details away.

This means we need to define new symbolic operations: and symbolic execution is now very simple:

But we now need to handle vadd op in `interp_op`, which defines how symbolic ops correspond to actual machine state operations. like with the first approach, this is easily modularized because of the slice, binop, `set_slice` pattern which vector operations all follow.

We define a helper function similar to the one used in the first approach: